Report CW1

Task 2.2: Code Analysis

The experiment compares the assembly generated for `add_vmlib()` and handwritten `add_ref()` using GCC 14.1 in Compiler Explorer, investigating compiler optimisations and the zero-cost abstraction principle.

Debug Mode

GCC uses a separate call to the overloaded `operator+`, demonstrating the absence of function inlining and introducing call overhead.

call SmallVec<4ul, float> operator+<4ul, float>(SmallVec<4ul, float> const&, SmallVec<4ul, float> const&)

The `vaddss` instruction performs scalar single-precision addition, processing one pair of 32-bit floats at a time. (Instructionsets.com, 2026)

vaddss xmm2, xmm0, DWORD PTR [rbp-52]

Release Mode

In release mode, by inlining operator+, GCC removes the need for a separate function call. Additionally, it makes use of vaddps, which uses a 128-bit xmm register to execute four simultaneous floating-point additions with SIMD.(2026, Instructionsets.com) As a result, there are fewer arithmetic instructions implemented.

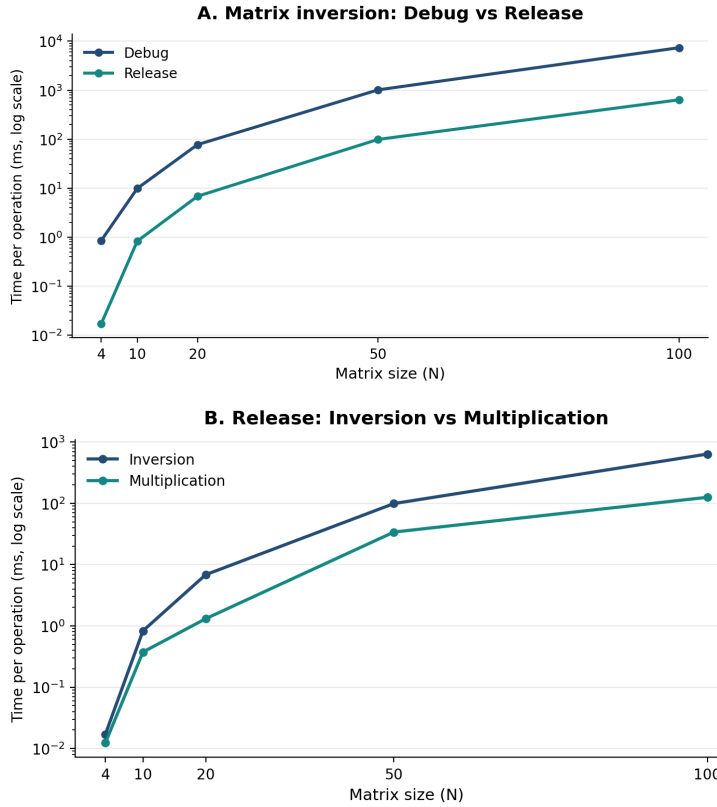
Library vs Handwritten Implementation

Both implementations generate nearly identical optimised assembly; both use `vmovups` for memory operations and `vaddps` for SIMD addition. Minor differences do not add computational overhead, supporting the zero-cost abstraction principle.

```
add_vmlib(unsigned long,
SmallVec<4ul, float>*, SmallVec<4ul,
float> const*, SmallVec<4ul, float>
const*):
test rdi, rdi
je .L8
sal rdi, 4
xor eax, eax
.L3:
vmovups xmm0, XMMWORD PTR [rcx+rax]
vaddps xmm0, xmm0, XMMWORD PTR
[rdx+rax]
vmovups XMMWORD PTR [rsi+rax], xmm0
add rax, 16
cmp rdi, rax
jne .L3
.L8:
ret
```

```
add_ref(unsigned long, RefVec*,
RefVec const*, RefVec const*):
test rdi, rdi
je .L17
sal rdi, 4
xor eax, eax
.L12:
vmovups xmm0, XMMWORD PTR [rdx+rax]
vaddps xmm0, xmm0, XMMWORD PTR
[rcx+rax]
vmovups XMMWORD PTR [rsi+rax], xmm0
add rax, 16
cmp rax, rdi
jne .L12
.L17:
ret
```

Task 2.3: Performance



Performance Analysis

Release mode is significantly faster than debug due to compiler optimisations. Both operations become slower as matrix size increases, with results consistent with approximately O(N³) scaling. Inversion is consistently slower than multiplication.

4×4 Analysis (special case)

Release inversion took 16.89 ns, whereas multiplication took 12.28 ns. The specific 4x4 inverse improves performance by eliminating general Gaussian elimination. Release inversion was about 50× faster than debug, even though its measurement was marked as unstable. By designating the 4x4 release inversion (6.2% error) and 20x20 release multiplication (30.9% error) as unstable, Nanobench made these measurements unreliable.

Conclusion and Test Environment

Release mode significantly improved performance. Execution time increased rapidly as matrix size grew, consistent with O(N³) complexity. Tests ran on a GitHub Codespace,using an Intel Xeon Platinum 8573C, 2 logical CPUs and 7.8 GiB RAM. Virtualisation and CPU frequency scaling may affect accuracy.

Gen AI

Copilot was used as an assistive tool to explain C++ concepts, debug code and interpret compiler assembly in some cases.

References

Instructionsets.com. (2026). *Vaddps (x86)*. [online] Available at: https://instructionsets.com/x86/vaddps/ [Accessed 27 Oct. 2026].

Instructionsets.com. (2026). *VADDSS (x86)*. [online] Available at: https://instructionsets.com/x86/vaddss/ [Accessed 27 Oct. 2026].